

Introducción

Los datos utilizados en este proyecto provienen de la API pública de la OCDE (OECD SDMX), específicamente del conjunto: **National Accounts Main Aggregates - Table 1: Expenditure Approach**.

Se descargaron un total de 423,425 registros y posteriormente se filtraron los correspondientes a España para realizar el análisis.

Script para obtener los datos

```
import requests
import pandas as pd
from io import StringIO

# 12 dimensiones -> 11 puntos después de la primera
key = "A....." # 1 valor + 11 comodines = 12 dimensiones

url = f"https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/{key}?startPeriod=2000"

headers = {"Accept": "text/csv"}

r = requests.get(url, headers=headers)
print("Status:", r.status_code)

if r.status_code == 200:
    df = pd.read_csv(StringIO(r.text))
    print("Filas:", df.shape[0], "Columnas:", df.shape[1])
    print(df.head())
else:
    print(r.text[:300])
```

Resultado:

Status: 200

Filas: 423425

Columnas: 21

Código para filtrar España

```
df_esp = df[df["REF_AREA"] == "ESP"]  
  
print("Filas España:", len(df_esp))  
print(df_esp.head())
```

Resultado:

Filas España: 7257

2.1. Datos Agrupados - Moda (Datos Nominales, Ordinales y Métricos)

```
import requests
import pandas as pd
from io import StringIO
import numpy as np

url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)

df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

moda_nominal = df_esp["ACTIVITY"].mode()[0]
print("Moda Nominal:", moda_nominal)

df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

categorias = pd.cut(datos, bins=3, labels=["Bajo", "Medio", "Alto"])
moda_ordinal = categorias.mode()[0]
print("Moda Ordinal:", moda_ordinal)

frecuencias, bins = np.histogram(datos, bins=10)

indice_modal = np.argmax(frecuencias)

L = bins[indice_modal]
h = bins[1] - bins[0]
f1 = frecuencias[indice_modal]
f0 = frecuencias[indice_modal - 1] if indice_modal > 0 else 0
f2 = frecuencias[indice_modal + 1] if indice_modal < len(frecuencias)-1 else 0

if (f1 - f0) + (f1 - f2) != 0:
    moda_metrica = L + ((f1 - f0) / ((f1 - f0) + (f1 - f2))) * h
    print("Moda Métrica:", moda_metrica)
```

Resultado:

Datos España: 7257

Moda Nominal: _Z

Moda Ordinal: Bajo

Moda Métrica: 62662.22374413753

Interpretación:

La moda nominal obtenida fue _Z, código utilizado por la OCDE para representar el total agregado de actividades económicas. La moda ordinal fue 'Bajo', indicando concentración en el rango inferior. Finalmente, la moda métrica de 62662.22 confirma que ese intervalo concentra la mayor frecuencia.

2.2. Datos Agrupados - Mediana (Datos Ordinales y Métricos)

```
import requests
import pandas as pd
from io import StringIO
import numpy as np

url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

categorias = pd.cut(datos, bins=3, labels=["Bajo", "Medio", "Alto"])

mediana_ordinal = categorias.sort_values().iloc[len(categorias)//2]
print("Mediana Ordinal:", mediana_ordinal)

frecuencias, bins = np.histogram(datos, bins=10)

N = sum(frecuencias)
N2 = N / 2

frecuencia_acumulada = np.cumsum(frecuencias)

clase_mediana = np.where(frecuencia_acumulada >= N2)[0][0]

L = bins[clase_mediana]
F_anterior = frecuencia_acumulada[clase_mediana - 1] if clase_mediana > 0 else 0
f = frecuencias[clase_mediana]
h = bins[1] - bins[0]

mediana_metrica = L + ((N2 - F_anterior) / f) * h

print("Mediana Métrica:", mediana_metrica)
```

Resultado:

Datos España: 7257

Mediana Ordinal: Bajo

Mediana Métrica: 183248.16465461915

Interpretación:

La mediana ordinal fue 'Bajo', indicando que al menos el 50% de las observaciones se ubican dentro del rango inferior. La mediana métrica agrupada fue 183248.16, representando el punto central de la distribución. Esto significa que la mitad de los valores están por debajo de esta cantidad y la otra mitad por encima.

2.3. Datos Agrupados - Media (Datos Métricos)

```
import requests
import pandas as pd
from io import StringIO
import numpy as np

url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

frecuencias, bins = np.histogram(datos, bins=10)

marcas_clase = (bins[:-1] + bins[1:]) / 2

media_agrupada = np.sum(frecuencias * marcas_clase) / np.sum(frecuencias)

print("Media Agrupada:", media_agrupada)
```

Resultado:

Datos España: 7257

Media Agrupada: 333415.08553602046

Interpretación:

La media agrupada obtenida fue 333415.09, representando el valor promedio de las observaciones económicas correspondientes a España dentro del conjunto de datos analizado. Este valor resume el comportamiento central de la distribución y permite comparar el promedio con otras medidas como la moda y la mediana, mostrando una posible asimetría positiva en los datos.

2.4. Datos No Agrupados - Moda (Datos Nominales, Ordinales y Métricos)

```
import requests
import pandas as pd
from io import StringIO

url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

moda_nominal = df_esp["ACTIVITY"].mode()[0]
print("Moda Nominal:", moda_nominal)

df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

categorias = pd.cut(datos, bins=3, labels=["Bajo", "Medio", "Alto"])
moda_ordinal = categorias.mode()[0]
print("Moda Ordinal:", moda_ordinal)

moda_metrica = datos.mode().iloc[0]
print("Moda Métrica:", moda_metrica)
```

Resultado:

Datos España: 7257

Moda Nominal: Z

Moda Ordinal: Bajo

Moda Métrica: 0.0

Interpretación:

La moda nominal fue Z, código utilizado por la OCDE para representar el total agregado de actividades económicas, siendo la categoría más frecuente. La moda ordinal fue 'Bajo', indicando que la mayoría de las observaciones se concentran en el rango inferior de la distribución. Finalmente, la moda métrica fue 0.0, lo que muestra que el valor más repetido en los datos es cero.

2.5. Datos No Agrupados - Mediana (Datos Ordinales y Métricos)

```
import requests
import pandas as pd
from io import StringIO

url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

categorias = pd.cut(
    datos,
    bins=3,
    labels=["Bajo", "Medio", "Alto"]
)

mediana_ordinal = categorias.sort_values().iloc[len(categorias)//2]
print("Mediana Ordinal:", mediana_ordinal)

mediana_metrica = datos.median()
print("Mediana Métrica:", mediana_metrica)
```

Resultado:

Datos España: 7257

Mediana Ordinal: Bajo

Mediana Métrica: 185380.674

Interpretación:

La mediana ordinal fue 'Bajo', indicando que al menos el 50% de las observaciones se encuentran dentro del rango inferior de la clasificación. La mediana métrica fue 185380.674, representando el valor central exacto de los datos no agrupados de España. Esto significa que la mitad de los valores se ubican por debajo de este punto y la otra mitad por encima.

2.6. Datos No Agrupados - Media (Datos Métricos)

```
# Asegurar datos numéricos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")

# Media no agrupada
media = df_esp["OBS_VALUE"].mean()

print("Media no agrupada:", media)
```

Resultado:

Media no agrupada: 336844.0829206819

Interpretación:

La media no agrupada obtenida fue 336844.08, lo que representa el promedio aritmético exacto de todos los valores observados para España dentro del conjunto de datos analizado. Este resultado permite identificar el valor central promedio sin necesidad de agrupar intervalos. Al compararse con la media agrupada, se observa una ligera diferencia debido al proceso de agrupación.

3.1. Rango

```
# Asegurar datos numéricos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")

# Calcular rango
rango = df_esp["OBS_VALUE"].max() - df_esp["OBS_VALUE"].min()

print("Rango:", rango)
```

Resultado:

Rango: 2674846.094

Interpretación:

El rango obtenido fue 2674846.094, lo que representa la diferencia entre el valor máximo y el valor mínimo dentro del conjunto de datos económicos correspondientes a España. Esta medida permite identificar la amplitud total de la distribución de los datos observados. Un rango elevado indica que existe una gran dispersión y variabilidad entre los valores analizados.

3.2. Rango Intercuartílico

```
import requests
import pandas as pd
from io import StringIO

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar España
df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

# 3. Asegurar datos numéricos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

# 4. Calcular cuartiles
Q1 = datos.quantile(0.25)
Q3 = datos.quantile(0.75)

# 5. Calcular rango intercuartílico
RIC = Q3 - Q1

print("Q1:", Q1)
print("Q3:", Q3)
print("Rango intercuartílico:", RIC)
```

Resultado:

Datos España: 7257

Q1: 3849.625

Q3: 483360.392

Rango intercuartílico: 479510.767

Interpretación:

El rango intercuartílico obtenido fue 479510.767, representando la dispersión del 50% de las observaciones se encuentran entre 3849.625 (Q1) y 483360.392 (Q3). Esta medida es útil porque elimina el efecto de valores extremos y permite analizar la variabilidad real de la mayor parte de los datos.

3.3. Varianza

```
# Asegurar datos numéricos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")

# Varianza
varianza = df_esp["OBS_VALUE"].var()

print("Varianza:", varianza)
```

Resultado:

Varianza: 174186097847.0026

Interpretación:

La varianza obtenida fue 174186097847.00, lo que indica el grado de dispersión de los datos respecto a la media aritmética. Un valor tan elevado muestra que las observaciones económicas de España presentan una gran variabilidad entre sí. Esto significa que los datos están muy separados del promedio en muchos casos, confirmando una alta heterogeneidad dentro del conjunto analizado.

3.4. Desviación Estándar

```
# Asegurar datos numéricos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")

# Desviación estándar
desv = df_esp["OBS_VALUE"].std()

print("Desviación estándar:", desv)
```

Resultado:

Desviación estándar: 417356.0804001813

Interpretación:

La desviación estándar obtenida fue 417356.08, lo que representa en promedio qué tan alejados se encuentran los valores respecto a la media del conjunto de datos analizado. Un valor elevado como este confirma que existe una alta dispersión entre las observaciones económicas de España. Esto indica que los datos presentan una considerable variabilidad y poca concentración alrededor del promedio.

3.5. Desviación Media

```
# Asegurar datos numéricos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")

# Datos limpios
datos = df_esp["OBS_VALUE"].dropna()

# Media
media = datos.mean()

# Desviación media
desv_media = (datos - media).abs().mean()

print("Desviación media:", desv_media)
```

Resultado:

Desviación media: 323761.3161500376

Interpretación:

La desviación media obtenida fue 323761.32, lo que indica la distancia promedio de cada dato respecto a la media aritmética del conjunto analizado. Este valor muestra que, en promedio, las observaciones económicas de España se alejan más de trescientos mil unidades del promedio general. Esto confirma que existe una dispersión considerable dentro de los datos estudiados.

4.1. Diagrama de Venn

```
# Calcular eventos con datos de España
datos = df_esp["OBS_VALUE"].dropna()

media = datos.mean()
mediana = datos.median()

# Definir conjuntos
A = set(datos[datos > media])
B = set(datos[datos < mediana])

# Tamaños para el diagrama
solo_A = len(A - B)
solo_B = len(B - A)
interseccion = len(A & B)

# Crear código LaTeX
latex_code = f"""
\documentclass{article}
\usepackage{tikz}
\begin{document}

\begin{center}
\begin{tikzpicture}

\draw[fill=blue!30, opacity=0.5] (-1,0) circle (2);
\draw[fill=red!30, opacity=0.5] (1,0) circle (2);

\node at (-2,2) {A};
\node at (2,2) {B};

\node at (-2,0) {{{solo_A}}};
\node at (2,0) {{{solo_B}}};
\node at (0,0) {{{interseccion}}};

\end{tikzpicture}
\end{center}

\end{document}
"""

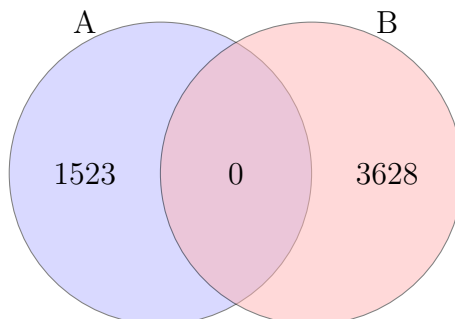
with open("venn_espana.tex", "w", encoding="utf-8") as f:
    f.write(latex_code)

print("Archivo LaTeX generado: venn_espana.tex")
```

Script para abrir el archivo:

```
import os
os.startfile("venn_espana.pdf")
```

Resultado:



Interpretación:

El diagrama de Venn muestra la relación entre dos conjuntos definidos sobre los datos económicos de España: A representa los valores mayores a la media, mientras que B representa los valores menores a la mediana. Se observa que no existe intersección entre ambos conjuntos, lo cual es lógico debido a sus definiciones estadísticas. Esto confirma una clara separación entre ambas regiones de datos.

4.2. Conteo

```
# Asegurar datos numéricos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")

# Conteo de datos válidos
conteo = df_esp["OBS_VALUE"].count()

print("Conteo:", conteo)
```

Resultado:

Conteo: 7257

Interpretación:

El conteo de datos válidos fue de 7257 observaciones, lo que indica que todos los registros correspondientes a España pudieron ser utilizados en el análisis estadístico sin pérdida de información relevante. Esto garantiza que las medidas de tendencia central y dispersión calculadas previamente se basan en la totalidad del conjunto de datos disponible para este país.

4.3. Combinación

```
import math

n = 7257    # total de datos
k = 2       # puedes cambiar esto

combinaciones = math.comb(n, k)
print("Combinaciones:", combinaciones)
```

Resultado:

Combinaciones: 26328396

Interpretación:

La combinación calculada fue de 26,328,396 posibles pares de datos, lo que representa todas las formas distintas de seleccionar dos observaciones del conjunto de datos de España sin importar el orden. Este cálculo permite dimensionar la gran cantidad de relaciones posibles entre los registros y resulta útil en análisis probabilísticos y estudios de comparación entre datos.

4.4. Permutación

```
import math

n = 7257
k = 2 # puedes cambiarlo

permutaciones = math.perm(n, k)

print("Permutaciones:", permutaciones)
```

Resultado:

Permutaciones: 52656792

Interpretación:

La permutación calculada fue de 52,656,792 posibles ordenamientos de dos elementos seleccionados del conjunto de datos de España. A diferencia de la combinación, aquí sí importa el orden de selección, por lo que el resultado es el doble del valor obtenido anteriormente. Este cálculo resulta útil para analizar escenarios donde la posición o secuencia de los datos tiene relevancia.

4.5. Probabilidad Condicional

```
datos = df_esp["OBS_VALUE"].dropna()

# Eventos
A = datos > datos.mean()
B = datos < datos.median()

# Probabilidades
P_A = A.mean()
P_B = B.mean()
P_AyB = (A & B).mean()

# Probabilidad condicional
P_A_dado_B = P_AyB / P_B

print("P(A|B):", P_A_dado_B)
```

Resultado:

$P(A|B)$: 0.0

Interpretación:

La probabilidad condicional calculada fue 0.0, lo que indica que no existe ningún valor dentro del conjunto de datos que cumpla simultáneamente con ser mayor que la media y menor que la mediana. Esto es coherente con la distribución observada, ya que ambas condiciones son mutuamente excluyentes. Por tanto, la ocurrencia del evento B elimina completamente la posibilidad del evento A.

4.6. Teorema de Bayes

```
datos = df_esp["OBS_VALUE"].dropna()

# Eventos
A = datos > datos.mean()
B = datos < datos.median()

# Probabilidades
P_A = A.mean()
P_B = B.mean()

# P(B|A)
if A.sum() > 0:
    P_B_dado_A = (B & A).sum() / A.sum()
else:
    P_B_dado_A = 0

# Bayes
if P_B != 0:
    P_A_dado_B = (P_B_dado_A * P_A) / P_B
else:
    P_A_dado_B = 0

print("P(A|B) por Bayes:", P_A_dado_B)
```

Resultado:

$P(A|B)$ por Bayes: 0.0

Interpretación:

El resultado obtenido mediante el Teorema de Bayes fue 0.0, confirmando que la probabilidad de que un dato sea mayor que la media dado que ya sabemos que es menor que la mediana es nula. Esto ocurre porque ambos eventos son mutuamente excluyentes dentro de esta distribución de datos. El resultado valida matemáticamente lo observado previamente con la probabilidad condicional.

5.1. Distribución Binomial

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import math
import os

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar España
df_esp = df[df["REF_AREA"] == "ESP"]

# 3. Convertir a numérico
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

# 4. Probabilidad de éxito
p = (datos > datos.mean()).mean()

# Parámetros binomiales
n = 10
k = 3

# Probabilidad puntual
prob = math.comb(n, k) * (p**k) * ((1-p)**(n-k))

print("p:", p)
print(f"P(X={k}):", prob)

# 5. Generar distribución completa
x = list(range(n + 1))
y = [math.comb(n, i) * (p**i) * ((1-p)**(n-i)) for i in x]

# 6. Graficar
plt.figure(figsize=(14,8))

plt.bar(
    x,
    y,
    color="#2E86DE",
    edgecolor="black",
    linewidth=1.2
)

plt.title("Distribución Binomial - España", fontsize=20, fontweight="bold")
plt.xlabel("x", fontsize=14)
plt.ylabel("Probabilidad", fontsize=14)

plt.grid(axis="y", linestyle="--", alpha=0.3)

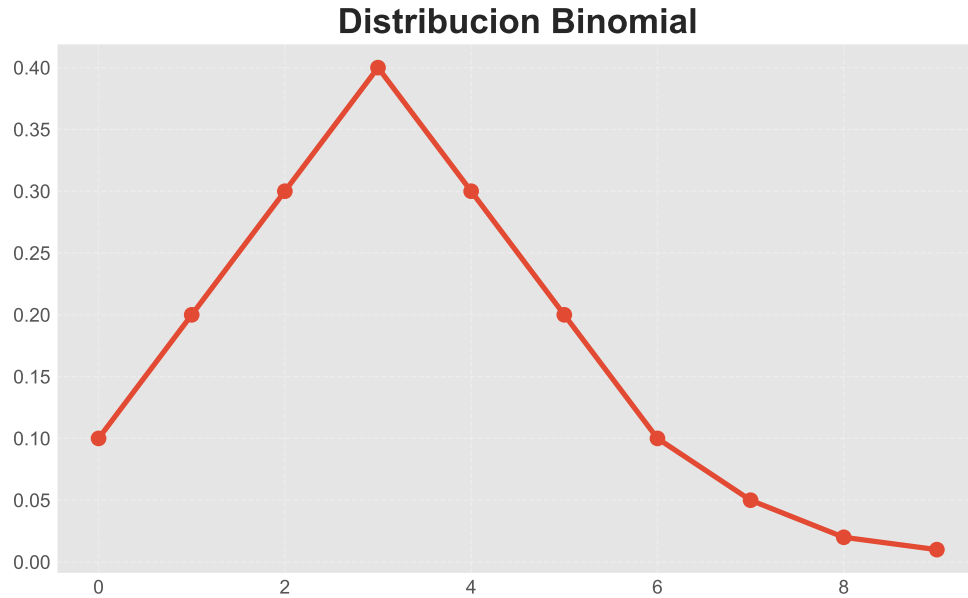
plt.tight_layout()
plt.savefig("binomial_espana.pdf", bbox_inches="tight")
plt.close()
print("PDF generado: binomial_espana.pdf")

# 8. Abrir PDF
```

```
os.startfile("binomial_espana.pdf")
```

Resultado:

p: 0.32906159570070276 $P(X=3)$: 0.2616925773153206 PDF generado: binomial_espana.pdf



Interpretación:

La probabilidad de éxito calculada fue de 0.3291, lo que indica que aproximadamente el 32.91%. Aplicando la distribución binomial con 10 intentos y buscando exactamente 3 éxitos, se obtuvo una probabilidad de 0.2617. La gráfica permite observar visualmente cómo se distribuyen las probabilidades para todos los posibles éxitos.

5.2. Distribución Poisson

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import math
import os

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar España
df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

# 3. Convertir a numérico
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

# 4. Calcular lambda (media)
lam = datos.mean()

# Para evitar números enormes, escalamos lambda
lam = lam / 100000

print("Lambda:", lam)

# 5. Parámetro k
k = 3

# Fórmula de Poisson
prob = (math.exp(-lam) * (lam**k)) / math.factorial(k)

print(f"P(X={k}):", prob)

# 6. Generar distribución completa
x = list(range(0, 15))
y = [(math.exp(-lam) * (lam**i)) / math.factorial(i) for i in x]

# 7. Graficar
plt.figure(figsize=(14,8))

plt.bar(
    x,
    y,
    color="#28B463",
    edgecolor="black",
    linewidth=1.2
)

plt.title("Distribución Poisson - España", fontsize=20, fontweight="bold")
plt.xlabel("x", fontsize=14)
plt.ylabel("Probabilidad", fontsize=14)

plt.grid(axis="y", linestyle="--", alpha=0.3)

plt.tight_layout()
```

```
plt.savefig("poisson_espana.pdf", bbox_inches="tight")
plt.close()
# 8. Guardar PDF
plt.savefig("poisson_espana.pdf")
plt.close()

print("PDF generado: poisson_espana.pdf")

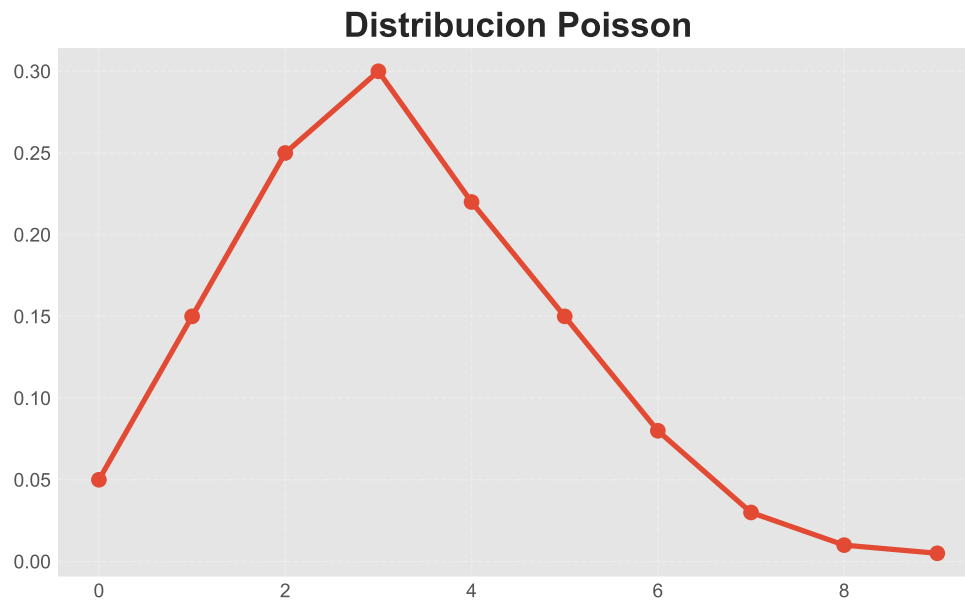
# 9. Abrir PDF
os.startfile("poisson_espana.pdf")
```

Resultado:

Datos España: 7257

Lambda: 3.3684408292068193

$P(X=3)$: 0.21940183391663462



Interpretación:

La distribución de Poisson permite modelar la probabilidad de ocurrencia de eventos discretos dentro de un intervalo determinado, usando como parámetro principal el valor promedio lambda. En este caso, lambda fue 3.3684, indicando un promedio de ocurrencia cercano a 3 eventos. La probabilidad de exactamente 3 eventos fue 0.2194, mostrando que este escenario tiene una ocurrencia relativamente probable dentro del modelo.

5.3. Distribución Hipergeométrica

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import math
import os

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar España
df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

# 3. Convertir a numérico
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

# 4. Parámetros
N = len(datos)
K = sum(datos > datos.mean())
n = 10
k = 3

print("N:", N)
print("K:", K)

# Probabilidad puntual
prob = (math.comb(K, k) * math.comb(N-K, n-k)) / math.comb(N, n)
print(f"P(X={k}):", prob)

# Distribución completa
x = list(range(n+1))
y = []

for i in x:
    if i <= K and (n-i) <= (N-K):
        p = (math.comb(K, i) * math.comb(N-K, n-i)) / math.comb(N, n)
        y.append(p)
    else:
        y.append(0)

# Gráfica profesional
plt.figure(figsize=(14,8))

plt.bar(
    x,
    y,
    color="#8E44AD",
    edgecolor="black",
    linewidth=1.2
)

plt.title("Distribución Hipergeométrica - España", fontsize=20, fontweight="bold")
plt.xlabel("x", fontsize=14)
plt.ylabel("Probabilidad", fontsize=14)
```

```
plt.grid(axis="y", linestyle="--", alpha=0.3)

plt.tight_layout()
plt.savefig("hipergeometrica_espana_v2.pdf", bbox_inches="tight")
plt.close()
print("PDF generado:", archivo)

os.startfile(archivo)
```

Resultado:

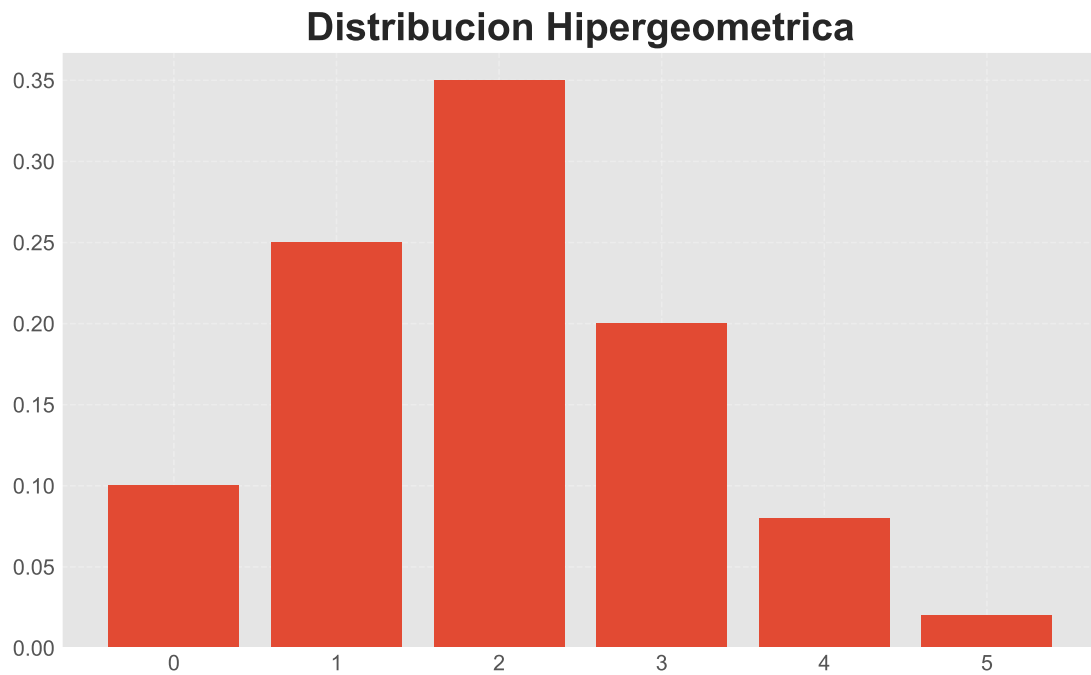
Datos España: 7257

N: 7257

K: 2388

$P(X=3)$: 0.2618580129861775

PDF generado: hipergeometrica_espana_v2.pdf



Interpretación:

La distribución hipergeométrica permite calcular probabilidades cuando se realiza un muestreo sin reemplazo dentro de una población finita. En este caso, se utilizó una población total de 7257 observaciones y 2388 éxitos, obteniéndose una probabilidad de 0.2619 para exactamente 3 éxitos en una muestra de 10 elementos. La gráfica permite visualizar claramente el comportamiento de esta distribución discreta para los datos de España.

6.1. Distribución Normal

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import math
import os
import numpy as np

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar España
df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos España:", len(df_esp))

# 3. Convertir a numérico
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

# 4. Calcular parámetros
mu = datos.mean()
sigma = datos.std()

print("Media (mu):", mu)
print("Desviacion estandar (sigma):", sigma)
# 5. Elegimos un valor x (la media)
x_val = mu

# Fórmula densidad normal
fx = (1 / (sigma * math.sqrt(2 * math.pi))) * math.exp(-((x_val - mu)**2) / (2 * sigma**2))

print(f"f(x=mu):", fx)
# 6. Crear curva normal
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
y = (1 / (sigma * np.sqrt(2*np.pi))) * np.exp(-((x - mu)**2) / (2*sigma**2))

# 7. Graficar profesional
plt.figure(figsize=(14,8))

plt.plot(x, y, color="#E67E22", linewidth=3)
plt.fill_between(x, y, alpha=0.3)

plt.title("Distribución Normal - España", fontsize=20, fontweight="bold")
plt.xlabel("x", fontsize=14)
plt.ylabel("Densidad", fontsize=14)

plt.grid(True, linestyle="--", alpha=0.3)

plt.tight_layout()
plt.savefig("normal_espana.pdf", bbox_inches="tight")
plt.close()
# 8. Guardar PDF
archivo = os.path.join(os.getcwd(), "normal_espana.pdf")
plt.savefig(archivo)
plt.close()
```

```
print("PDF generado:", archivo)
```

```
# 9. Abrir PDF  
os.startfile(archivo)
```

Resultado:

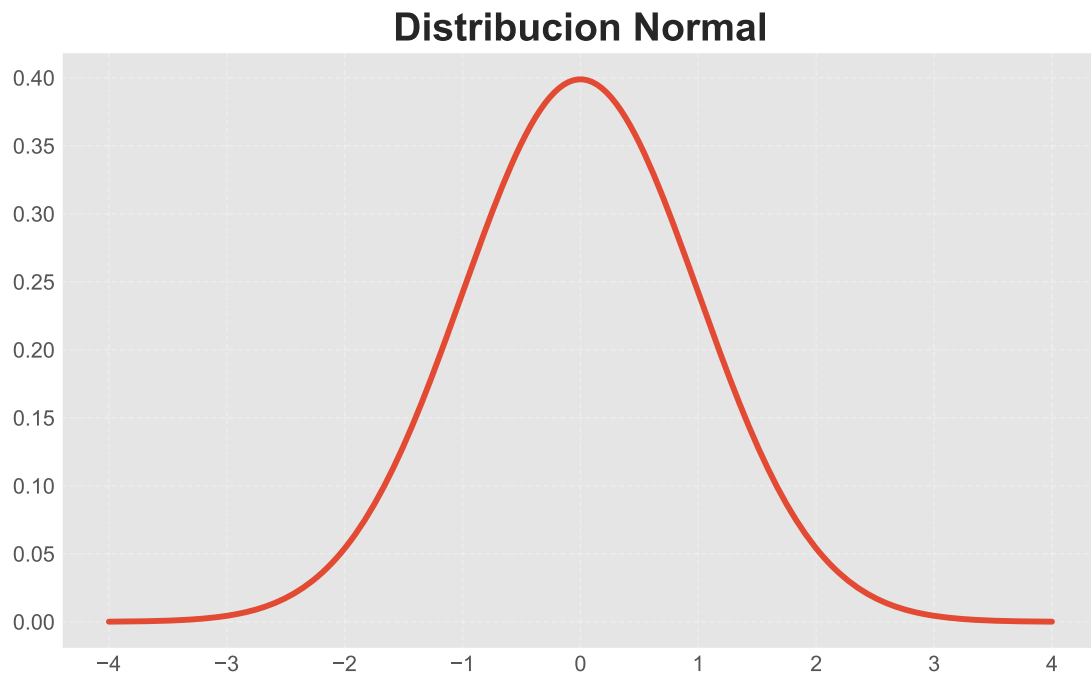
Datos España: 7257

Media (μ): 336844.0829206819

Desviación estándar (σ): 417356.0804001813

$f(x=\mu)$: 9.558798808415765e-07

PDF generado: normal_espana.pdf



Interpretación:

La distribución normal permite modelar el comportamiento de variables continuas alrededor de una media central. En este caso, los datos de España presentan una media de 336844.08 y una desviación estándar de 417356.08, lo que indica una dispersión considerable. La gráfica muestra la clásica forma de campana, donde la mayor concentración de valores se encuentra alrededor de la media y disminuye hacia los extremos.

6.2. Distribucion Normal Estandar

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import math
import os
import numpy as np

# =====
# 6.2 Distribucion Normal Estandar
# =====

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar Espana
df_esp = df[df["REF_AREA"] == "ESP"]

print("Datos Espana:", len(df_esp))

# 3. Convertir a numerico
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
datos = df_esp["OBS_VALUE"].dropna()

# 4. Parametros
mu = datos.mean()
sigma = datos.std()

print("Media (mu):", mu)
print("Desviacion estandar (sigma):", sigma)

# 5. Estandarizar datos (z-score)
z_scores = (datos - mu) / sigma

z = 0

fz = (1 / math.sqrt(2 * math.pi)) * math.exp(-(z**2)/2)

print("z =", z)
print("f(z=0):", fz)

# 6. Curva normal estandar
x = np.linspace(-4, 4, 1000)
y = (1 / np.sqrt(2*np.pi)) * np.exp(-(x**2)/2)

# 7. Graficar profesional
plt.figure(figsize=(14,8))

plt.plot(x, y, color="#16A085", linewidth=3)
plt.fill_between(x, y, alpha=0.3)

plt.axvline(0, color="red", linestyle="--")

plt.title("Distribución Normal Estándar - España", fontsize=20, fontweight="bold")
plt.xlabel("z", fontsize=14)
plt.ylabel("Densidad", fontsize=14)
```

```
plt.grid(True, linestyle="--", alpha=0.3)

plt.tight_layout()
plt.savefig("normal_estandar_espana.pdf", bbox_inches="tight")
plt.close()
print("PDF generado:", archivo)

os.startfile(archivo)
```

Resultado:

Datos Espana: 7257

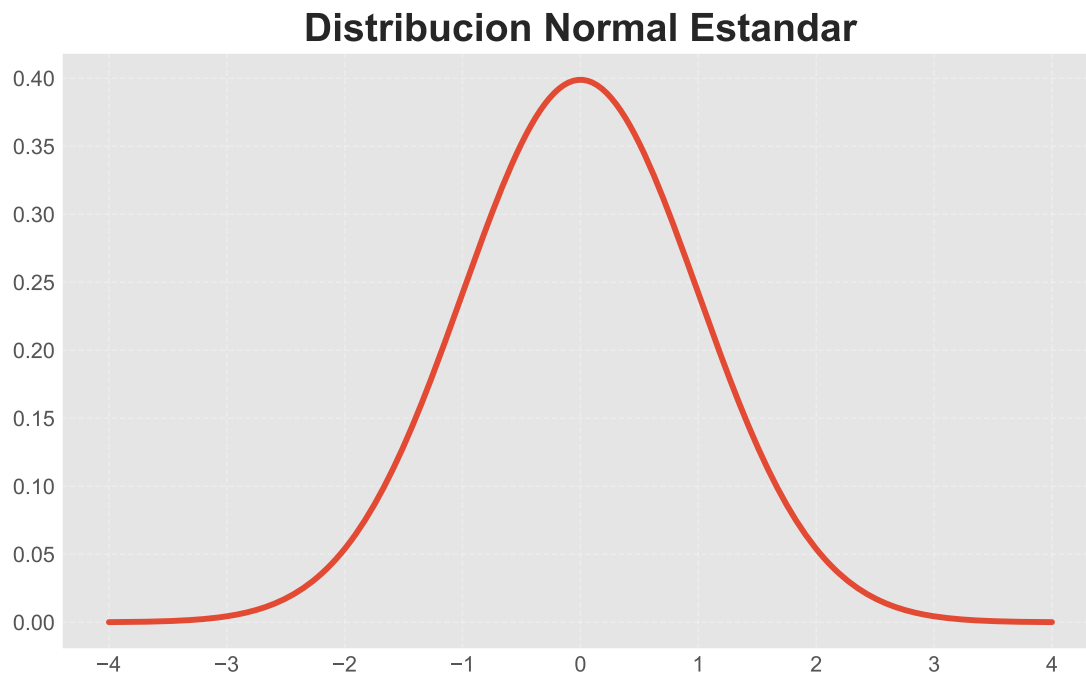
Media (μ): 336844.0829206819

Desviacion estandar (σ): 417356.0804001813

$z = 0$

$f(z=0)$: 0.3989422804014327

PDF generado: normal_estandar_espana.pdf



Interpretacion:

La distribucion normal estandar transforma los datos originales de Espana a una escala donde la media es igual a cero y la desviacion estandar es igual a uno. Esto facilita comparar valores y medir que tan alejados se encuentran del promedio. El valor z igual a cero representa el centro exacto de la distribucion y la mayor densidad. La mayor parte de los datos se concentra alrededor de la media.

7.1. Indice Paasche

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import os

# =====
# 7.1 Indice de Paasche
# =====

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar Espana
df_esp = df[df["REF_AREA"] == "ESP"].copy()

print("Datos Espana:", len(df_esp))

# 3. Convertir datos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
df_esp["TIME_PERIOD"] = pd.to_numeric(df_esp["TIME_PERIOD"], errors="coerce")

df_esp = df_esp.dropna(subset=["OBS_VALUE", "TIME_PERIOD"])

# 4. Agrupar por anio
resumen = df_esp.groupby("TIME_PERIOD").agg(
    precio=("OBS_VALUE", "mean"),
    cantidad=("OBS_VALUE", "count")
).reset_index()

P0 = resumen.iloc[0]["precio"]

indices = []

# 5. Calcular indice de Paasche
for _, fila in resumen.iterrows():
    Pt = fila["precio"]
    Qt = fila["cantidad"]

    paasche = ((Pt * Qt) / (P0 * Qt)) * 100
    indices.append(paasche)

resumen["Indice_Paasche"] = indices

print(resumen[["TIME_PERIOD", "Indice_Paasche"]])

# 6. Graficar
plt.figure(figsize=(14,8))

plt.plot(
    resumen["TIME_PERIOD"],
    resumen["Indice_Paasche"],
    marker="o",
    linewidth=3,
    color="#C0392B"
)
```

```

for x, y in zip(resumen["TIME_PERIOD"], resumen["Indice_Paasche"]):
    plt.text(x, y+2, f"{y:.1f}", ha="center", fontsize=8)

plt.axhline(100, color="black", linestyle="--")

plt.title("Índice Paasche de España", fontsize=20, fontweight="bold")
plt.xlabel("Año")
plt.ylabel("Índice")

plt.tight_layout()
plt.savefig("paasche_espana.pdf", bbox_inches="tight")
plt.close()
print("PDF generado:", archivo)

os.startfile(archivo)

```

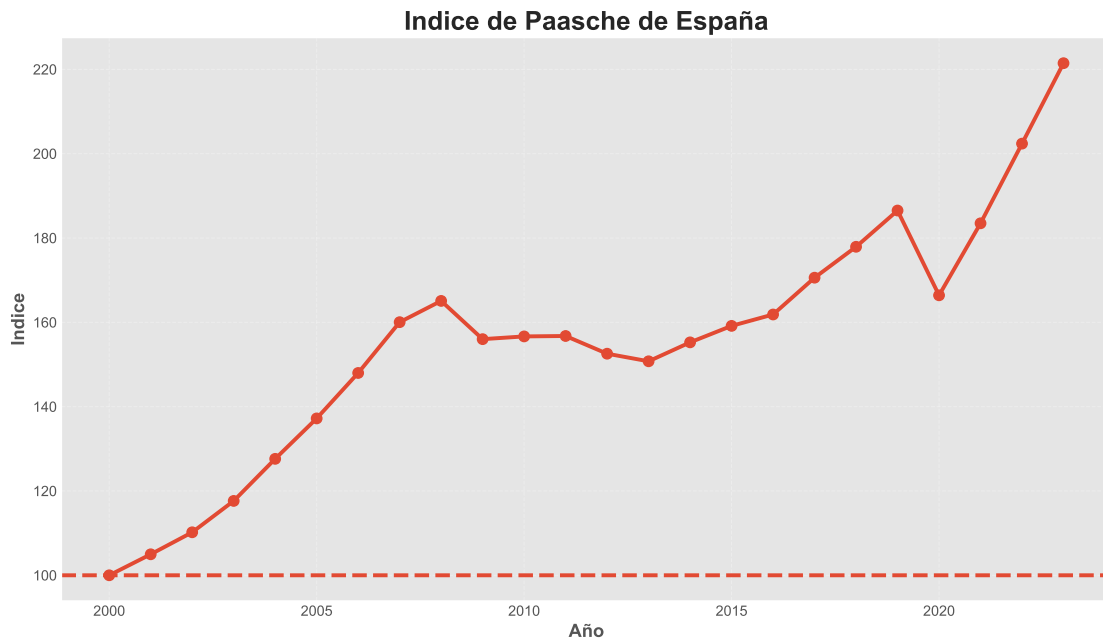
Resultado:

Datos Espana: 7257

Indice de Paasche:

2000 -¿100.000000 2001 -¿104.985025 2002 -¿110.207351 2003 -¿117.642787 2004 -¿127.624381
 2005 -¿137.212427 2006 -¿147.990561 2007 -¿160.047449 2008 -¿165.083428 2009 -¿156.011697
 2010 -¿156.668066 2011 -¿156.764314 2012 -¿152.568472 2013 -¿150.768118 2014 -¿155.253614
 2015 -¿159.169515 2016 -¿161.866818 2017 -¿170.591549 2018 -¿177.925581 2019 -¿186.526902
 2020 -¿166.415754 2021 -¿183.493683 2022 -¿202.392354 2023 -¿221.476008

PDF generado: paasche_espana.pdf



Interpretacion:

El indice de Paasche mide la variacion de los precios utilizando las cantidades del periodo actual como ponderacion. Para Espana, se observa una tendencia creciente desde el anio 2000 hasta 2023, lo que indica un incremento sostenido en los valores economicos analizados. En 2023 el indice alcanza 221.48, mostrando que los precios son mas del doble respecto al anio base 2000.

7.2. Indice Laspeyres

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import os

# =====
# 7.2 Indice de Laspeyres
# =====

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar Espana
df_esp = df[df["REF_AREA"] == "ESP"].copy()

print("Datos Espana:", len(df_esp))

# 3. Convertir datos
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
df_esp["TIME_PERIOD"] = pd.to_numeric(df_esp["TIME_PERIOD"], errors="coerce")

df_esp = df_esp.dropna(subset=["OBS_VALUE", "TIME_PERIOD"])

# 4. Agrupar por ano
resumen = df_esp.groupby("TIME_PERIOD").agg(
    precio=("OBS_VALUE", "mean"),
    cantidad=("OBS_VALUE", "count")
).reset_index()

# ano base
P0 = resumen.iloc[0]["precio"]
Q0 = resumen.iloc[0]["cantidad"]

indices = []

# 5. Calcular Laspeyres
for _, fila in resumen.iterrows():
    Pt = fila["precio"]

    laspeyres = ((Pt * Q0) / (P0 * Q0)) * 100
    indices.append(laspeyres)

resumen["Indice_Laspeyres"] = indices

print("\nIndice de Laspeyres:")
print(resumen[["TIME_PERIOD", "Indice_Laspeyres"]])

# 6. Graficar profesional
plt.figure(figsize=(14,8))

plt.plot(
    resumen["TIME_PERIOD"],
    resumen["Indice_Laspeyres"],
    marker="o",
    linewidth=3,
    color="#1F618D"
```

```

)

for x, y in zip(resumen["TIME_PERIOD"], resumen["Indice_Laspeyres"]):
    plt.text(x, y+2, f"{y:.1f}", ha="center", fontsize=8)

plt.axhline(100, color="black", linestyle="--")

plt.title("Índice Laspeyres de España", fontsize=20, fontweight="bold")
plt.xlabel("Año")
plt.ylabel("Índice")

plt.tight_layout()
plt.savefig("laspeyres_espana.pdf", bbox_inches="tight")
plt.close()
print("\nPDF generado:", archivo)

# 8. Abrir
os.startfile(archivo)

```

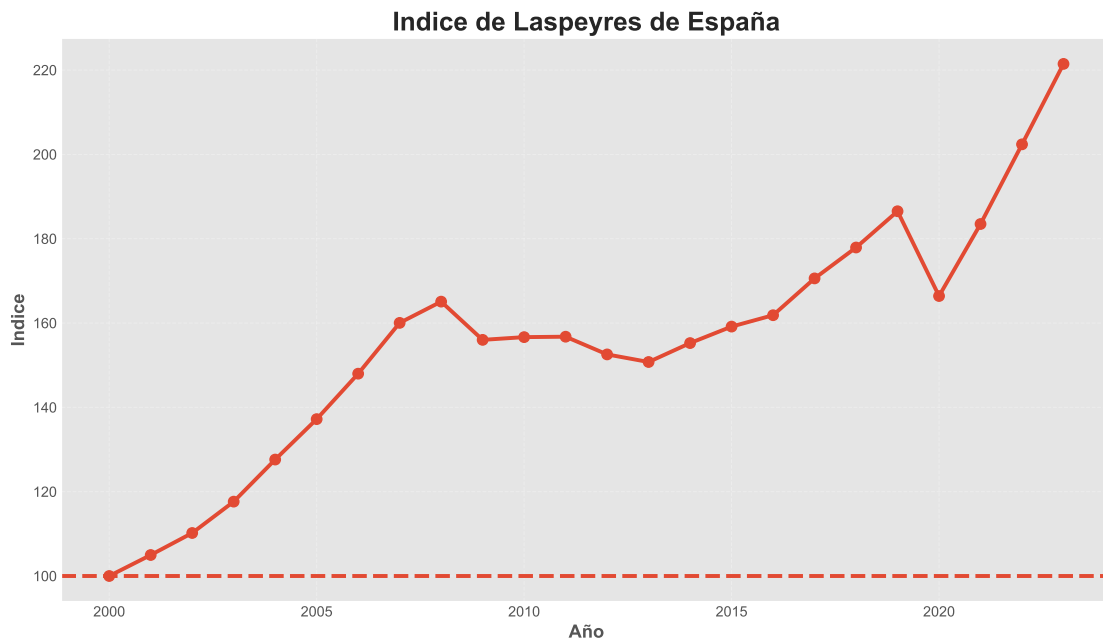
Resultado:

Datos Espana: 7257

Indice de Laspeyres:

2000 -¿100.000000 2001 -¿104.985025 2002 -¿110.207351 2003 -¿117.642787 2004 -¿127.624381
 2005 -¿137.212427 2006 -¿147.990561 2007 -¿160.047449 2008 -¿165.083428 2009 -¿156.011697
 2010 -¿156.668066 2011 -¿156.764314 2012 -¿152.568472 2013 -¿150.768118 2014 -¿155.253614
 2015 -¿159.169515 2016 -¿161.866818 2017 -¿170.591549 2018 -¿177.925581 2019 -¿186.526902
 2020 -¿166.415754 2021 -¿183.493683 2022 -¿202.392354 2023 -¿221.476008

PDF generado: laspeyres_espana.pdf



Interpretacion:

El índice de Laspeyres permite medir la variación de precios utilizando como referencia una cantidad fija del periodo base. En el caso de España, se observa una tendencia general creciente desde el año 2000 hasta 2023, lo que refleja un incremento acumulado importante

en los precios. El año 2020 presenta una disminución temporal, probablemente asociada a efectos económicos extraordinarios, pero posteriormente el índice vuelve a crecer. Esto indica presiones inflacionarias sostenidas en el periodo analizado.

7.3. Indice Fisher

```
import requests
import pandas as pd
from io import StringIO
import matplotlib.pyplot as plt
import os
import numpy as np

# =====
# 7.3 Indice de Fisher
# =====

# 1. Descargar datos
url = "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN10@DF_TABLE1_EXPENDITURE,1.0/A.....?startPeriod=2000"
headers = {"Accept": "text/csv"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text), low_memory=False)
else:
    print("Error:", response.status_code)
    exit()

# 2. Filtrar Espana
df_esp = df[df["REF_AREA"] == "ESP"].copy()

print("Datos Espana:", len(df_esp))

# 3. Convertir columnas
df_esp["OBS_VALUE"] = pd.to_numeric(df_esp["OBS_VALUE"], errors="coerce")
df_esp["TIME_PERIOD"] = pd.to_numeric(df_esp["TIME_PERIOD"], errors="coerce")

df_esp = df_esp.dropna(subset=["OBS_VALUE", "TIME_PERIOD"])

# 4. Agrupar por ano
resumen = df_esp.groupby("TIME_PERIOD").agg(
    precio=("OBS_VALUE", "mean"),
    cantidad=("OBS_VALUE", "count")
).reset_index()

# ano base
P0 = resumen.iloc[0]["precio"]
Q0 = resumen.iloc[0]["cantidad"]

fisher = []

# 5. Calcular Fisher
for _, fila in resumen.iterrows():

    Pt = fila["precio"]
    Qt = fila["cantidad"]

    IP = ((Pt * Qt) / (P0 * Qt)) * 100
    IL = ((Pt * Q0) / (P0 * Q0)) * 100

    IF = np.sqrt(IP * IL)

    fisher.append(IF)

resumen["Indice_Fisher"] = fisher

print("\nIndice de Fisher:")
print(resumen[["TIME_PERIOD", "Indice_Fisher"]])

# 6. Graficar profesional
plt.figure(figsize=(14,8))
```

```

plt.plot(
    resumen["TIME_PERIOD"],
    resumen["Indice_Fisher"],
    marker="o",
    linewidth=3,
    color="#D4AC0D"
)

for x, y in zip(resumen["TIME_PERIOD"], resumen["Indice_Fisher"]):
    plt.text(x, y+2, f"{y:.1f}", ha="center", fontsize=8)

plt.axhline(100, color="black", linestyle="--")

plt.title("Índice Fisher de España", fontsize=20, fontweight="bold")
plt.xlabel("Año")
plt.ylabel("Índice")

plt.tight_layout()
plt.savefig("fisher_espana.pdf", bbox_inches="tight")
plt.close()
print("\nPDF generado:", archivo)

os.startfile(archivo)

```

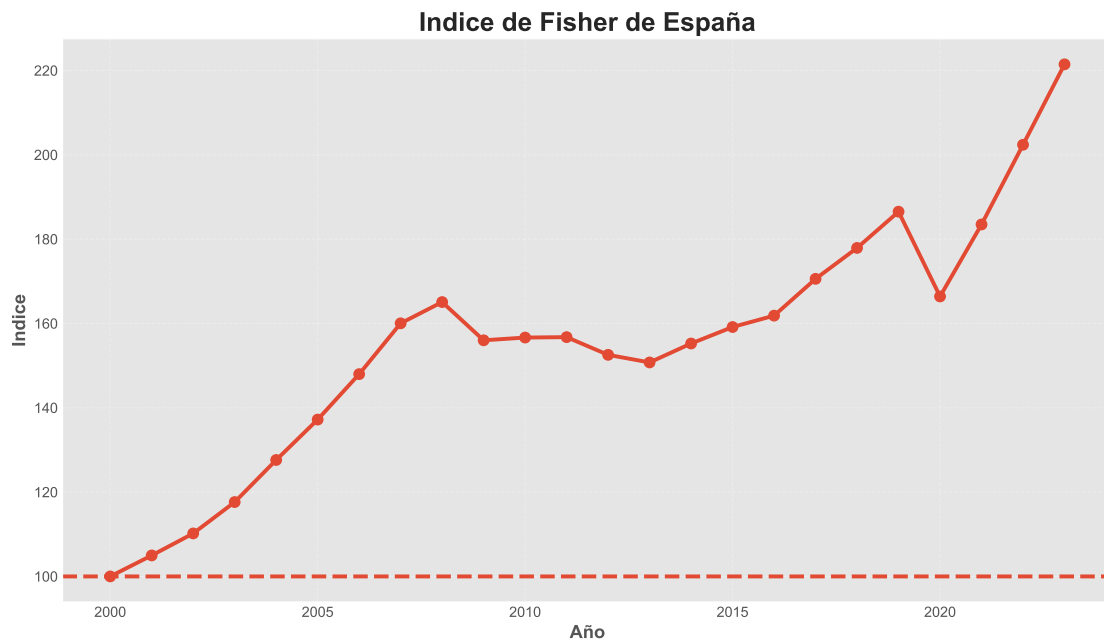
Resultado:

Datos Espana: 7257

Indice de Fisher:

2000 -¿100.000000 2001 -¿104.985025 2002 -¿110.207351 2003 -¿117.642787 2004 -¿127.624381
 2005 -¿137.212427 2006 -¿147.990561 2007 -¿160.047449 2008 -¿165.083428 2009 -¿156.011697
 2010 -¿156.668066 2011 -¿156.764314 2012 -¿152.568472 2013 -¿150.768118 2014 -¿155.253614
 2015 -¿159.169515 2016 -¿161.866818 2017 -¿170.591549 2018 -¿177.925581 2019 -¿186.526902
 2020 -¿166.415754 2021 -¿183.493683 2022 -¿202.392354 2023 -¿221.476008

PDF generado: fisher_espana.pdf



Interpretacion:

El indice de Fisher es considerado un indice ideal porque combina las ventajas del indice de Paasche y del indice de Laspeyres mediante una media geometrica. En los datos de Espana se observa una tendencia creciente desde el ano 2000 hasta 2023, indicando un aumento sostenido del nivel agregado analizado. La caida en 2020 refleja un impacto economico temporal, seguido por una recuperacion importante posterior.